QCon
全球软件开发大会

InfoQ 技术大会

AI 在研发组织落地的三段式路径

从人使用 AI 干活，到 AI 自主交付

李佳奇

去哪儿旅行 基础架构负责人 技术总监



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 研发团队落地 AI 的困境和解决思路

02 三步走其一：先解决“看得清”

03 三步走其二：从个人提效到组织产出

04 三步走其三：让 AI 自主干活

05 总结：推进建议和展望

01

困境与解决思路

AI 能力 ≠ 组织生产力

困境：AI 能力 ≠ 组织生产力

过去两年，模型和工具的能力一路狂飙，但多数研发组织的交付结果并没有同步变快。问题不在能力侧，在落地侧。

AI 能力侧（供给）

模型能力每几个月上一个台阶

编码助手、Agent 工具人人可及

单点任务效率提升肉眼可见

≠

组织生产力侧（结果）

需求交付周期没有明显变短

质量与返工没有明显改善

投入产出比，谁也说不清楚

个人用得爽 ≠ 组织变强了。中间隔着的这道鸿沟，就是这次分享要拆开的东西。

坑一：把“使用率高”当成“落地成功”

数据涨了，交付没变快——这是我们走的第一个弯路。

1. **现象** 使用率、出码率月月上涨，汇报材料非常好看；但需求交付周期、线上质量纹丝不动。

2. **为什么** 使用率、出码率是过程指标，测的是“忙不忙”，不是“有没有产出”；一旦和考核挂钩，大家优化的是指标本身。

3. **后果** 管理层误判形势，以为落地已经成功，错过真正的改进窗口；预算花了，信心却被透支。

考核什么，就会得到什么——考核使用率，得到的就只是使用率。

坑二：提效停留在员工个人电脑里

个人省下的时间，组织收不回来——这是第二个坑。

个人视角

写代码快了，查资料快了

每个人都真切感到“省时间”

满意度调查一片好评

→?

组织视角

省下的时间没有变成更多交付

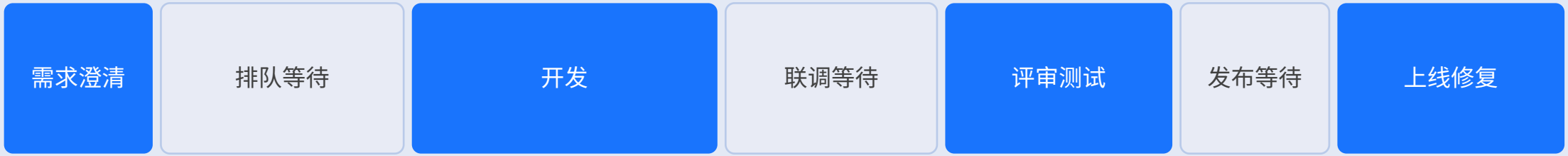
被等待、评审排队、返工吃掉

流程不变，红利在交接处蒸发

个人提效是“势能”，组织流程不改，势能变不成“动能”。

拆开一个需求的旅程，时间到底花在哪

把需求从提出到上线的全程拉直看：真正“动手处理”的时间，往往只是一小部分。（示意结构）



蓝色是处理时间，浅色是等待时间。 只优化“开发”一段，等于只动了全程的一小块；等待和交接吃掉的时间，工具再强也省不回来。

所以第二步的方向不是“让开发更快”， 而是让每个环节、每次交接都有 AI 参与，把等待和返工一起压下去。

先看清时间的结构，再决定 AI 应该铺在哪里。

坑三：AI 在干什么，看不见

无法衡量，就无法管理——这是最基础、也最要命的一个坑。

看不见用量

谁在用、用在哪个环节

消耗了多少 token

没有统一账本，各算各的

看不见产出

AI 写的代码进了哪些需求

质量如何、返工多少

无从追溯，更无从优化

看不见风险

无效消耗藏在黑盒里

敏感代码、合规风险不可见

好经验也无法复制推广

三个“看不见”，直接决定了我们的第一步：先解决“看得清”。

解决思路：AI 落地不是接入问题，是管理问题

接入问题（已解决）

买工具、发账号、做培训

解决的是“能不能用”

这一步必要，但远远不够

管理问题（待解决）

定目标：到底要什么结果

建度量：投入产出看得清

做运营：持续优化而不是一发了之

我们的路线图： 先度量，看得清 → 再嵌入业务流程 → 有边界地让 AI 自主交付

核心命题：无法度量 = 无法运营。AI 落地是一场组织变革，不是一次工具采购。

02

先解决“看得清”

给 AI 装上一套仪表盘

为什么度量要先于推广

没有度量就推广 AI，等于开盲盒：token 花在哪里、换来什么，全靠猜。

1. **代价一：预算失控** 用量涨得很快，但值不值没人说得清；预算评审时拿不出依据，投入随时可能被砍。

2. **代价二：争议无据** 业务方质疑投入产出时只能各说各话；信心在没有数据的时候最脆弱。

3. **代价三：优化无向** 不知道哪个环节、哪个团队用得好，好经验无法复制，问题也定位不到。

先推广、后度量，补课的成本远高于一开始就建好仪表盘。

度量体系：覆盖全公司、全研发流程

我们做的第一件事，是建立一套覆盖全公司、贯通研发全流程的 AI 效能度量体系。

数据层	指标层	应用层
工具调用数据 代码提交与评审数据 需求与交付数据	使用率 产出贡献率 交付结果	管理看板（多级视图） 运营抓手（推广/成本） 决策支持（投入与筛选）

建设原则：全公司统一口径、全流程贯通、一个数据账本——避免各团队各算各的。

关键指标（一）：使用率与产出贡献率

使用率

回答：有没有在用

活跃使用人数、覆盖团队数

看渗透深度：用了哪些能力

用法：看趋势，不搞绝对值排名

产出贡献率

回答：AI 产出了多少真实成果

AI 生成代码进入主干的占比

AI 参与的需求 / 任务占比

AI 产出被采纳、被保留的比例

只看使用率会被刷；只看贡献率会逼人硬塞 AI——两层必须一起看。

产出贡献率怎么落地：AI 代码的识别与口径

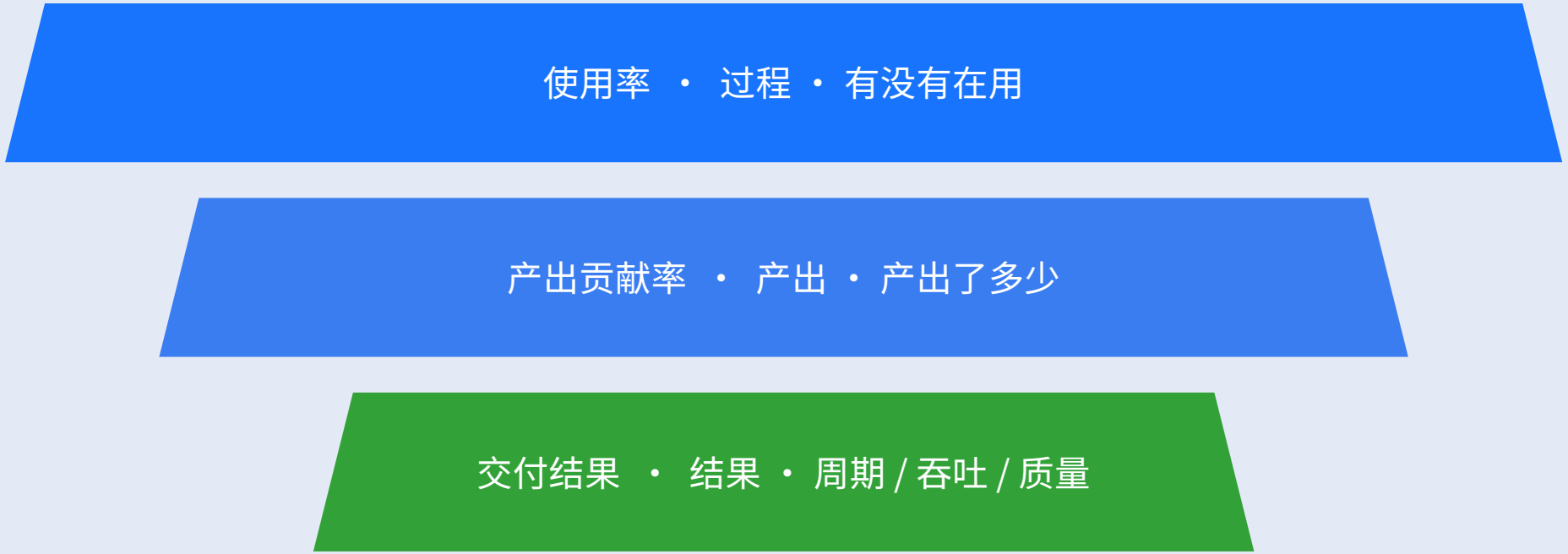
贡献率最难的一步，是回答“这段代码算不算 AI 产出”。口径不定，数就不可信。

工具侧标记	**从源头打标** 编码工具与 Agent 在生成时留下标记，提交时自动带入——覆盖最全，但依赖工具支持与统一接入。
提交规范	**靠统一工具** 研发团队全员安装统一数据采集工具，通过工具下发标准并保证执行
抽样校准	**用样本修正全局** 定期人工抽样标注，对全量数据做偏差修正——解决“标记不全”和“数据不实”两个方向的误差。

没有完美的识别方法：目标是口径稳定、趋势可信，而不是逐行精确。

关键指标（二）：交付结果

三层指标越往下，越接近真实价值，也越难归因——这是必须诚实面对的。



归因的态度： 交付受多因素影响，不把改善全记在 AI 头上；看趋势、看试点与非试点的对照，不追求精确归因。汇报时三层一起给，单看一层都会误导。

度量建好之后：运营节奏比看板更重要

看板挂在那里没人看，等于没有度量。度量要变成运营动作，必须有固定节奏。

周 · 看趋势抓 case

团队负责人看每周需求，搭配分析工具逐个分析归因低出码率、低使用率，落实到具体开发人员

→

双周 · 横向拉齐

AI 研发 SIG 双周通晒各团队数据，共享优秀方法论和工具，交流优秀案例

→

季 · 调方向

按进展调整下季度方向，从追求出码率到研发渗透率再到端到端和职能融合

↺ 闭环：复盘的结论回到投入决策，下一轮接着跑

度量的终点不是一张报表，而是一串按节奏发生的决策。

一条红线：度量是为组织决策，不是为个人排名

度量一旦变成对个人的监控和排名，数据立刻开始说谎——这是坑一的加强版。

- 看团队，不看个人：指标的最小颗粒度是团队，个人数据只用于本人自查
- 看趋势，不搞排名：排名制造刷量，趋势才帮助改进
- 先沟通，后使用：口径和用途对团队透明，数据不做“突袭式”考核
- 保留团队自主性：各团队可根据自身情况和目标设置，做定制化指标，保留自主解释权利

实践现状：度量体系落地情况

2025 Q4 - 2026 Q2 看出码率和使用率



出码率快速拉升，AI 工具在研发团队中快速普及

实践现状：度量体系落地情况

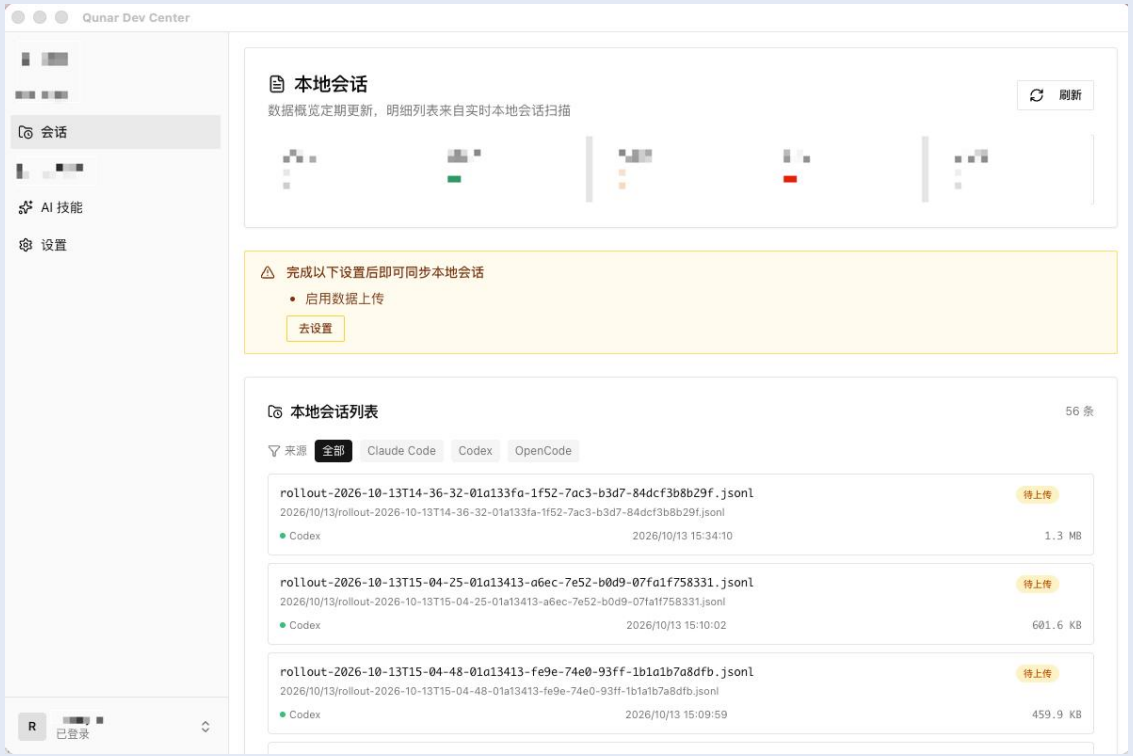
2025 Q4 - 2026 Q2 看出码率和使用率

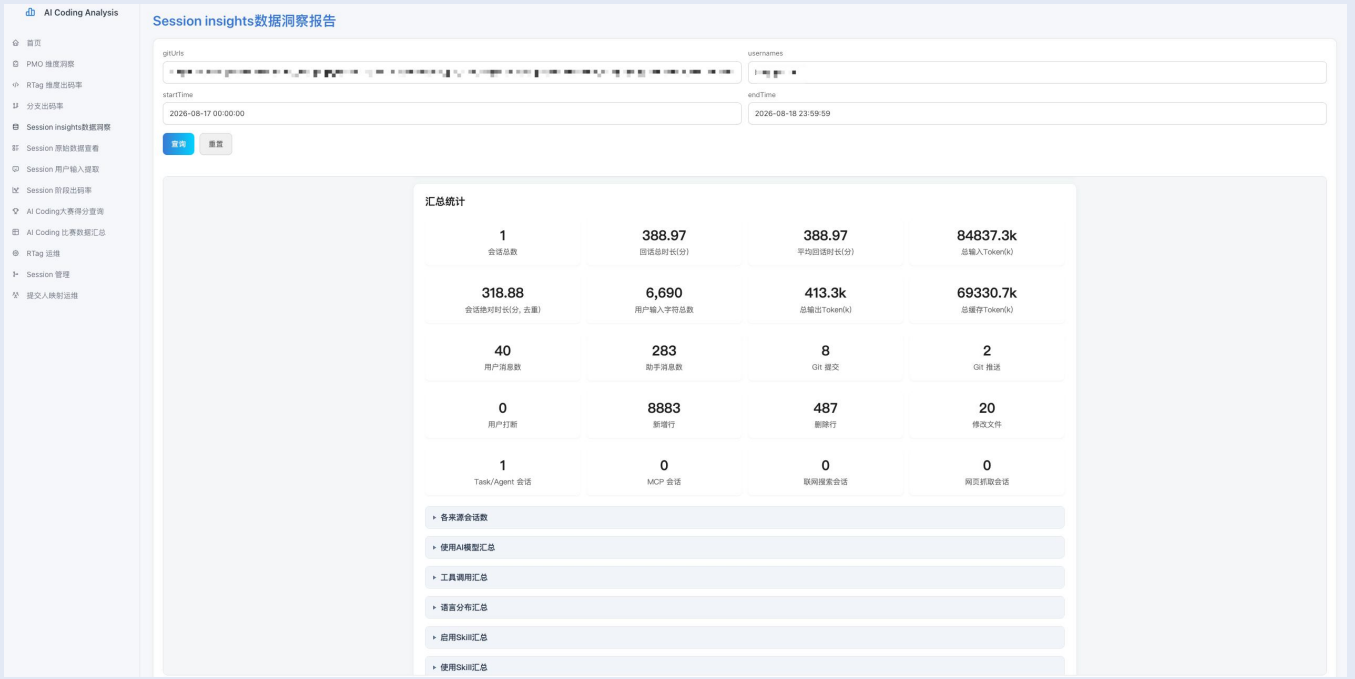


需求率快速提升，研发人员在完成需求的过程中使用 AI 工具

实践现状：度量体系落地情况

公司级 AI 研发度量工具和洞察平台

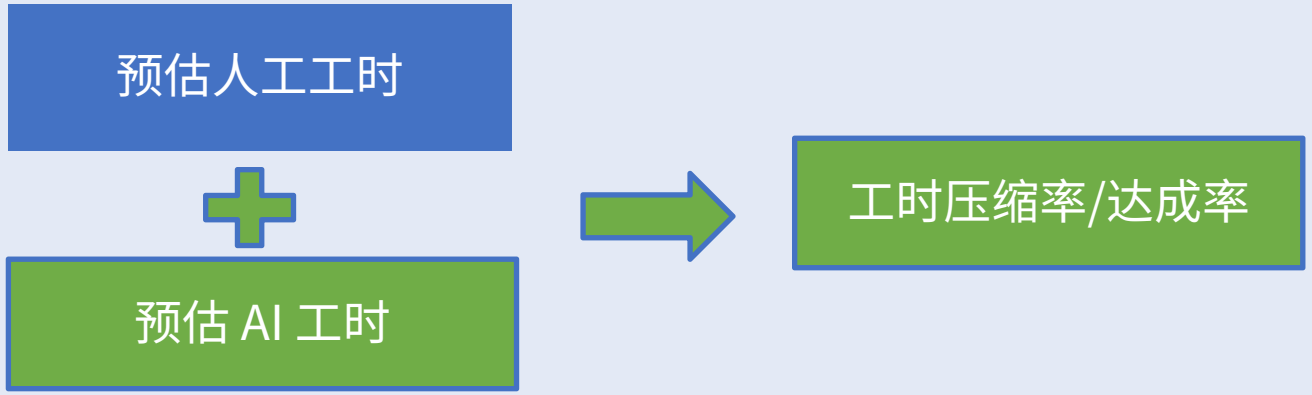




工具解决全流程会话采集，保证数据真实；洞察平台基于真实数据持续分析 AI 研发过程，挖掘提升空间

实践现状：度量体系落地情况

研发效能变化度量：双估时 + 达成率





研发效能口径设计需要找到对比指标，否则变化会被质疑是否真实有效

03

从个人提效到组织产出

让 AI 嵌入完整研发流程

为什么个人提效会被“稀释”

约束理论早就给过答案：系统吞吐由瓶颈决定，一个环节提速，总周期取决于新的瓶颈。

瓶颈转移

编码快了

评审、测试、联调成为新瓶颈

提速的环节开始排队等待

交接损耗

需求澄清、上下文传递

返工与对齐会议

把省下的时间原样吃回去

局部最优陷阱

人人优化自己的环节

在制品越堆越多

全局周期不降反升

要收走个人提效的红利，必须改流程，而不是继续给个人加工具。

把 AI 铺到研发全流程的每一个环节

我们的做法：沿着需求的生命周期逐环节渗透——不是每个环节都要 AI 主导，而是每个环节都不让 AI 缺席。

需求分析	技术设计	编码实现	代码评审	测试验证	发布上线	线上运维
需求澄清 历史需求检索 影响面分析	方案生成与对比 风险点提示 评审材料准备	编码助手 代码生成 单元测试生成	AI 预审 规范检查 安全扫描	用例生成 回归分析 缺陷定位	变更摘要 发布检查 回滚预案	日志分析 故障定位 值班助手

先渗透，再谈深度：全流程的水位整体抬升，比单点明星案例更重要。

深挖一个环节：代码评审的 AI 渗透

以代码评审为例，看一个环节被 AI 渗透前后的流程差异

之前：纯人工评审

提交后排队等评审人

评审人先花时间理解背景与改动意图

规范类问题占用大量评审注意力

深度问题反而容易被漏掉

之后：AI 预审 + 人工把关

提交即触发 AI 预审：规范、安全、常见缺陷先过一遍

评审人打开时已带摘要与风险点标注

人工注意力集中在设计与业务正确性

评审记录自动沉淀，可追溯、可度量

渗透的标志不是“用了 AI”，而是流程本身因为 AI 发生了重排。

渗透是动态过程：瓶颈会搬家

每个环节提速之后，瓶颈会移动到下一个环节——渗透不是一次性工程，要用度量持续追踪。

第一阶段	需求	**开发（瓶颈）**	评审	测试
第二阶段	需求	开发	**评审（瓶颈）**	测试
第三阶段	**需求（瓶颈）**	开发	评审	**测试（瓶颈）**

瓶颈搬到哪里，下一轮渗透就跟到哪里——这是第一步的度量在持续指路。

组织配套动作：流程、平台、组织

把 AI 铺进流程，靠的不是号召，而是三件配套建设。

流程规范

AI 产出如何评审、如何验收、责任边界如何划定——先立规矩，再放量；没有规矩，团队不敢用、用乱了也没法管。

统一平台（数字化、运行时）

数字化已介绍过：工具、模型、账号、成本收口

避免多头采购与数据孤岛

平台同时是度量的数据源头

虚拟组织：AI 研发 SIG

横向推进机制与布道者网络

场景共创：和业务团队共建用法

把一线经验沉淀为全公司资产

流程给规矩，虚拟组织给人

工具配套动作：AI 工具从开发本地到平台化

将本地的 CC、Codex等工具和运行环境迁移到云平台，确保规范能落地，具备更好的研发流程融入能力，7 * 24 不间断交付

AI 研发云平台

规范的落地需要有平台的支撑，开发个人的 AI Loop 需要通过平台进行流程嵌入和分发，将个人提效转变为组织提效的核心基建

AI 研发云平台：天弦

CC、Codex等主流工具托管和高可用

专业的研发专家模板化配置

专业的研发运行时Sandbox定制

研发流程嵌入：如意项目助手

Jira 项目生命周期驱动

研发全流程各环节 AI 专家编排

无缝嵌入到产品、研发、运营协作流程中

平台给抓手

实践现状：研发流程渗透率指标

不再看 demo 级别的案例，而在全流程的“水位”整体抬升。

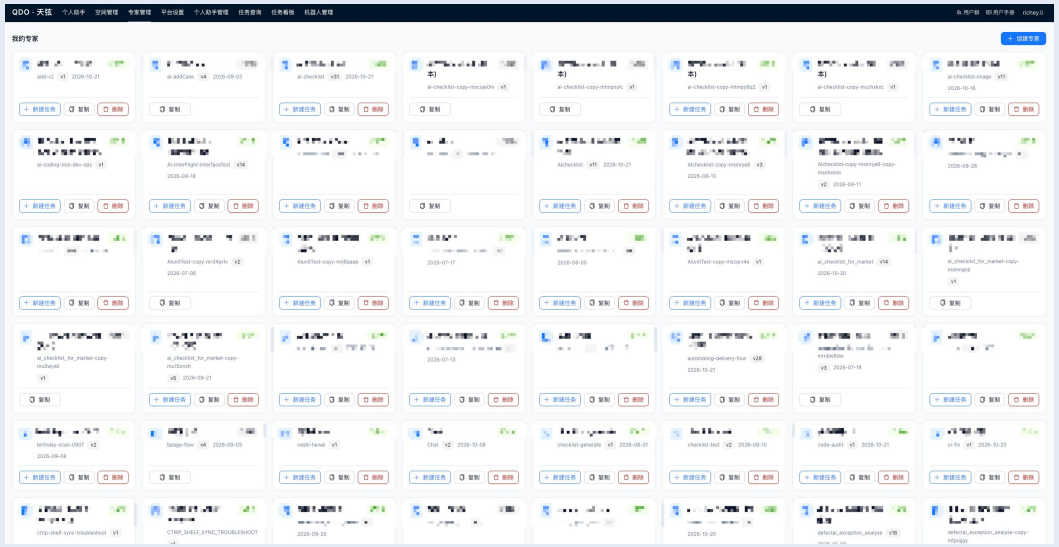




全司研发需求均纳入渗透率指标体系，团队级数据下钻到需求级，各团队针对性提升低渗透率指标

实践现状：AI 研发平台建设

AI 研发专家定义配置、从需求到运维的研发流程各阶段专家覆盖







平台上已有专家覆盖需求、设计、编码、联调、测试、发布全链路，已接管 20000+ 研发需求

实践现状：AI 研发流程的嵌入

通过 AI 项目助手嵌入到团队已有的研发流程和产品、研发、运营的协作流程







为公司每个需求项目标配 AI 助手，在产研运沟通飞书群中作为入口调度研发全流程中各个 AI 专家

04

让 AI 自主干活

端到端交付的边界与条件

老板真正想要的是什么

研发流程最终被压缩成一个点，从 idea 到交付的周期大大缩短，过程中人员能力的要求大幅降低，需要的专业职能大幅减少

研发流程被压缩

传统的从需求到发布的研发流程被压缩成一个AI节点

idea 到交付周期缩短

更快速的从机会到交付，带来更多次的出手和试错

人员能力要求降低

AI 沉淀几十年的行业领域能力，人员经过简单培训就可以上手

资深专家职能需求减少

垂直有深度的场景仅需要少数资深专家，AI 发挥杠杆作用

从终局倒推路径

什么场景适合让 AI 独立完成

不靠感觉，用度量数据回答——第二步沉淀的过程数据，就是这一步的场景筛选器。

结果指标变化显著

研发效能大幅提升、职能要求大幅降低

过程指标优秀

高出码率、高渗透率、高贡献率

交付流程明显变化

跨职能、跨团队、跨人员交接大幅减少

团队主动追求

团队自身重点投入的端到端交付领域

四个条件齐了，才谈自主交付；缺任何一个，就退回人机协作。

AI 自主交付，需要重新设计流程

老流程是为人设计的：靠默契补全、靠人在环中对齐。把执行者换成 AI，流程本身必须重做。

环节	老流程（为人设计）	自主交付流程（为 AI 设计）
任务输入	文档 / 口头交接，靠默契补全	规格先行：结构化任务描述与约束
执行过程	人在环中，边做边对齐	AI 独立执行，关键节点留痕
质量把关	人工评审、事后抽查	自动验收：测试与校验前置
协作方式	线上线下会议、私聊	在各个协作渠道中为AI设计角色

AI 自主交付需要重新确立职能需求

通过上一阶段的 AI 专家建设和渗透，使得交付流程中人员的职能发生变化，专业性要求降低，人的职责从执行到验收

- ✓ **需求专家** 需要产研运团队协同建设，在需求文档编写、优化等环节提升专家贡献率
- ✓ **开发专家** 研发团队最容易建设的职能，高出码率、低故障率是核心指标
- ✓ **测试专家** 最难建设的职能，人为交付质量最终负责
- ✓ **运维专家** 最容易被忽视环节，长尾问题极多

各个专家打磨的完备程度，直接决定自主交付敢走多远。

决策者的误区：方向对，不等于现在全量上

误区一：当口号

把“端到端”喊成 KPI

没有验收体系就放手

等于把风险放大到全流程

误区二：用案例定预期

演示场景的成功

替代不了生产场景的稳定性要求

预期管理一旦失真，反噬最快

误区三：一步到位

跳过人机协作阶段

团队信任没有积累过程

一次事故就整体崩塌

渐进路径： 人审（AI 做、人审） → 抽检（AI 做、按比例抽检） → 自主 + 监控（AI 做、指标监控、异常升级）

端到端是方向，但盲目推进的代价，远大于晚一步的收益损失。

通过数据确认端到端实现进度

结果条件（交付周期压缩 or 职能种类收敛 or 协作模式改变） + 过程条件（高出码率 and 高渗透率 and 高贡献度）+ 业务线主观标签（业务线 pmo 标签 或 jira 默认标签：端到端交付）



高渗透率



交付周期压缩



职能收敛



主观标签

通过之前介绍的数字化指标识别端到端，而不是头口表述

我们目前看到的各环节研发端到端交付的上限



如意项目演示

以 fastjson 漏洞修复为例

演示流程：PMO 创建 → 群聊触发 → 后台长链路执行 → 报告回写 → 发布上线

2026-09-09 v4

05

总结与展望

给决策者的推进建议

全景回顾：三步走路线图

回头看，三步走是一个整体：每一步的产出，都是下一步的输入。

	第一步 · 看得清	第二步 · 嵌入流程	第三步 · 自主交付
核心问题	AI 在干什么、值不值	个人提效如何变组织产出	哪些场景能放手让 AI 干
关键抓手	度量体系与三层指标	全流程渗透 + 流程/平台/组织配套	场景筛选 + 验收体系 + 晋级门槛
留给下一步	数据与口径：筛选场景的依据	过程数据与测试/规则资产	组织信任与可复制的方法论

跳过任何一步，后面的步子都会踩空——顺序本身就是方法论的一部分。

给组织决策者的三条建议

01

先度量，后推广

没有度量就推广，是开盲盒

先把仪表盘建起来

再谈规模化投入

02

当组织变革做

不当能力接入做

工具接入解决“能不能用”

流程、平台、组织一个都不能少

03

务实划定边界

全流程渗透是当下最扎实的价值

端到端是方向，用数据筛场景

循序渐进，不冒进

顺序比速度重要：度量 → 渗透 → 自主，每一步都在为下一步攒数据和攒信任。

展望：从人用 AI，到 AI 成为组织能力

度量演进

从“用了多少”走向“交付了多少价值”

度量体系从运营工具

变成决策基础设施

流程演进

研发流程会为 AI 重新设计一遍

不是把 AI 塞进老流程

而是流程长出新形态

组织演进

人的角色向定目标、划边界、做验收集中

AI 承担越来越多执行环节

管理者的杠杆点随之迁移

AI 落地的终局，不是谁的工具更强，而是谁的组织先完成这次变革。

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

从人使用 AI 干活，到 AI 自主交付

